

To create and activate a new isolated Conda environment named **env** with **Python version 3.8**

Software Requirements

- Anaconda or Miniconda installed on the system
- Supported operating systems: Windows / Linux / macOS
- Terminal or Anaconda Prompt access

Procedure

Step 1: Open Conda Terminal

- On **Windows**: Open *Anaconda Prompt*
- On **Linux/macOS**: Open the system terminal

Step 2: Create a New Environment

Execute the following command to create a Conda environment named env with Python 3.8:

```
conda create -n env python=3.8
```

- `-n env` specifies the environment name
- `python=3.8` ensures the required Python version

When prompted, type **y** and press **Enter** to proceed.

Step 3: Activate the Environment

Activate the newly created environment using:

```
conda activate env
```

Step 4: Verify Python Version

Confirm the Python version inside the environment:

```
python --version
```

Expected output:

```
Python 3.8.x
```

Result

A new Conda environment named **env** was successfully created and activated with **Python 3.8**, providing an isolated and controlled development environment.

Conclusion

The Conda environment ensures dependency isolation and version control, which is essential for stable software development, experimentation, and reproducibility.

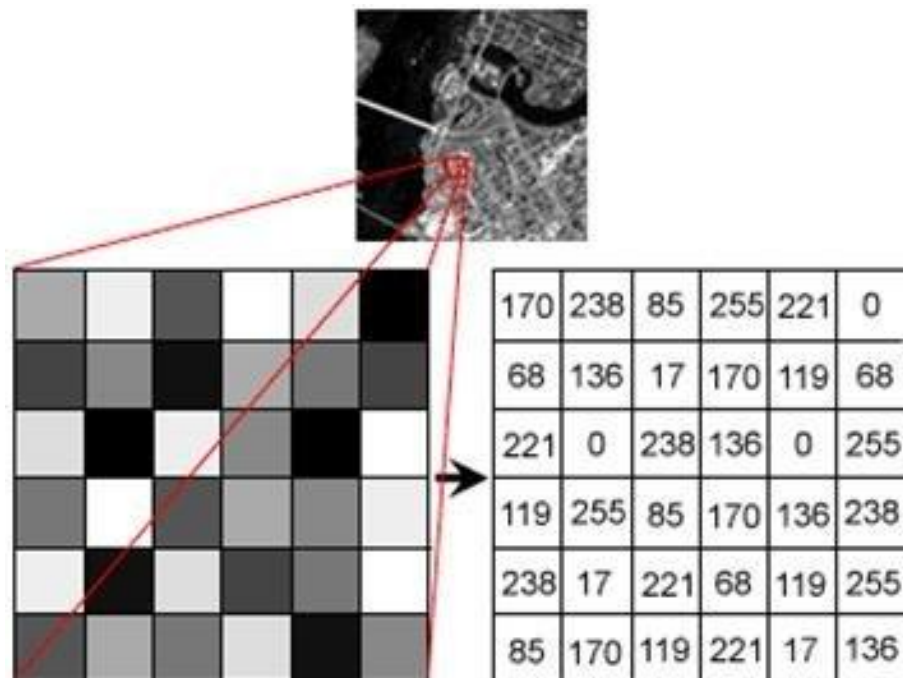
What is an image in OpenCV?

Before code, set the concept:

- An image is **not a picture**, it's a **matrix of numbers**
- Each pixel = intensity value
- OpenCV reads images in **BGR format** (not RGB)

For example:

- Grayscale image → 2D matrix
- Color image → 3D matrix (Height × Width × Channels)



To install opencv

Pip install opencv-python

Import required libraries

Explain that OpenCV uses **cv2** in Python.

```
import cv2
```

Reading an image using **cv2.imread()**

Basic syntax

```
image = cv2.imread("image.jpg")
```

Explain:

- "image.jpg" → image path
- image → NumPy array

Image reading modes

```
cv2.imread("image.jpg", 1)    # Color (default)
cv2.imread("image.jpg", 0)    # Grayscale
cv2.imread("image.jpg", -1)   # Unchanged (with alpha channel)
```

Displaying the image

Use OpenCV's window functions:

```
cv2.imshow("My Image", image)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

- imshow() → show image
- waitKey(0) → wait forever until key press
- destroyAllWindows() → close windows

Checking image properties

```
print(image.shape)    # (height, width, channels)
print(image.dtype)    # usually uint8
```

explanation:

(480, 640, 3) → 480 rows, 640 columns, 3 color channels

uint8 → values from 0 to 255

Reading and displaying a grayscale image

```
gray = cv2.imread("image.jpg", 0)

cv2.imshow("Grayscale Image", gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

“In OpenCV, an image is a NumPy array read using cv2.imread() and displayed using cv2.imshow().”

If you're using **OpenCV (cv2)** and just want to resize an image to **640 × 640**, this is the clean, standard way

```
import cv2

img = cv2.imread("input.jpg")
resized_img = cv2.resize(img, (640, 640))

cv2.imshow("Resized", resized_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Notes

- OpenCV uses **(width, height)** → (640, 640)
- This **forces** the image into a square
→ aspect ratio **will change** if the original isn't square

Intensity of an Image (after Grayscale)

Intensity = brightness of a pixel

- **0** → Black
- **255** → White
- Values in between → shades of gray

So after converting to grayscale:

- Each pixel has **ONE value**
- That value represents **intensity**

```
import cv2

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Access pixel intensity values

Intensity of a single pixel

```
intensity = gray[100, 200]
print(intensity)
```

- `gray[row, column]`
- Output is a number between **0–255**

Show intensity range of the whole image

```
print("Min intensity:", gray.min())
print("Max intensity:", gray.max())
```

Typical output:

```
Min intensity: 12
Max intensity: 245
```

how that grayscale is just numbers:

```
print(gray)
```

Mean intensity (overall brightness)

```
mean_intensity = gray.mean()
```

```
print("Mean intensity:", mean_intensity)
```

- Low mean → dark image
- High mean → bright image

RGB Intensity of an Image

First: what does RGB intensity mean?

In a **color image**, each pixel has **3 intensity values**:

- **R** → Red intensity
- **G** → Green intensity
- **B** → Blue intensity

Each channel value ranges from **0 to 255**.

So one pixel looks like:

[B, G, R] (in OpenCV!)

Read a color image

```
import cv2

img = cv2.imread("image.jpg")    # BGR image
    • OpenCV reads images in BGR, not RGB
    • Image shape → (H, W, 3)

print(img.shape)
```

Access RGB (BGR) intensity of a pixel

```
b, g, r = img[100, 200]
print("Blue:", b)
print("Green:", g)
print("Red:", r)
```

Example output:

Blue: 45

Green: 120

Red: 200

Display intensity range of each channel

```
print("Blue min/max:", img[:, :, 0].min(), img[:, :, 0].max())
print("Green min/max:", img[:, :, 1].min(), img[:, :, 1].max())
print("Red min/max:", img[:, :, 2].min(), img[:, :, 2].max())
```

Split RGB channels

```
b, g, r = cv2.split(img)

cv2.imshow("Blue Channel", b)
cv2.imshow("Green Channel", g)
cv2.imshow("Red Channel", r)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Mean RGB intensity (image-level)

```
mean_b = img[:, :, 0].mean()
mean_g = img[:, :, 1].mean()
mean_r = img[:, :, 2].mean()

print("Mean B:", mean_b)
print("Mean G:", mean_g)
print("Mean R:", mean_r)
```

Difference between grayscale & RGB intensity

Image Type	Intensity Values
Grayscale	1 value per pixel
RGB/BGR	3 values per pixel

Class 2

Create a blank image using NumPy

An image is just a **NumPy array**.

```
import numpy as np
```

```
import cv2
```

```
# Create a black image (height=400, width=600, 3 color channels)
```

```
img = np.zeros((400, 600, 3), dtype=np.uint8)
```

```
cv2.imshow("Black Image", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

- `np.zeros` → black image
- `dtype=np.uint8` → pixel range 0–255 (VERY important)

Create a white image

```
white_img = np.ones((400, 600, 3), dtype=np.uint8) * 255
```

```
cv2.imshow("White Image", white_img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Create a colored image (BGR format)

Note: OpenCV uses **BGR**, not RGB.

```
# Blue image
```

```
blue_img = np.zeros((400, 600, 3), dtype=np.uint8)
```

```
blue_img[:, :] = (255, 0, 0) # B, G, R
```

```
cv2.imshow("Blue Image", blue_img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Examples:

- (255, 0, 0) → Blue
- (0, 255, 0) → Green
- (0, 0, 255) → Red

Draw shapes on the created image

```
img = np.zeros((400, 600, 3), dtype=np.uint8)
```

```
# Line
```

```
cv2.line(img, (50, 50), (550, 50), (0, 255, 0), 2)
```

```
# Rectangle
```

```
cv2.rectangle(img, (100, 100), (300, 250), (255, 0, 0), 2)
```

```
# Filled Circle
```

```
cv2.circle(img, (450, 200), 50, (0, 0, 255), -1)
```

```
cv2.imshow("Shapes", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Create a gradient image

```
gradient = np.zeros((256, 256), dtype=np.uint8)
```

```
for i in range(256):
```

```
    gradient[:, i] = i
```

```
cv2.imshow("Gradient Image", gradient)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```


Put text on an image

```
img = np.zeros((300, 600, 3), dtype=np.uint8)
```

```
cv2.putText(  
    img,  
    "OpenCV with NumPy",  
    (50, 150),  
    cv2.FONT_HERSHEY_SIMPLEX,  
    1,  
    (0, 255, 255),  
    2  
)
```

```
cv2.imshow("Text Image", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Create a grayscale image using NumPy and display it

A grayscale image is a **2D NumPy array** where each value represents intensity (0–255).

Concept

- Shape → (height, width)
- No color channels

Code

```
import numpy as np
```

```
import cv2
```

```
gray_img = np.zeros((300, 400), dtype=np.uint8)
```

```
gray_img[:, :] = 150 # medium gray
```

```
cv2.imshow("Grayscale Image", gray_img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Create an image with left half black and right half white

We modify pixel values using **array slicing**.

Code

```
img = np.zeros((300, 600), dtype=np.uint8)

img[:, :300] = 0   # black upto 300 column

img[:, 300:] = 255 # white after 300 column

cv2.imshow("Half Black Half White", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Create an RGB image and convert it to grayscale

Answer

OpenCV stores images in **BGR**, so conversion uses cv2.COLOR_BGR2GRAY.

Code

```
img = np.zeros((300, 400, 3), dtype=np.uint8)

img[:] = (0, 0, 255) # Red image (BGR)

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

cv2.imshow("Color Image", img)

cv2.imshow("Grayscale Image", gray)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Draw diagonal lines on a blank image

Answer

Use cv2.line() to draw from corners.

Code

```
img = np.zeros((400, 400, 3), dtype=np.uint8)


cv2.line(img, (0, 0), (399, 399), (0, 255, 0), 2)

cv2.line(img, (399, 0), (0, 399), (0, 0, 255), 2)
```

```
cv2.imshow("Diagonal Lines", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Create an image with increasing intensity from top to bottom

Answer

Pixel intensity increases row-wise.

Code

```
img = np.zeros((256, 256), dtype=np.uint8)
```

```
for i in range(256):
```

```
    img[i, :] = i
```

```
cv2.imshow("Vertical Gradient", img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Write text and save the image to disk

cv2.imwrite() saves the image.

Code

```
img = np.zeros((200, 500, 3), dtype=np.uint8)
```

```
cv2.putText(
    img,
    "OpenCV Practical",
    (50, 120),
    cv2.FONT_HERSHEY_SIMPLEX,
    1,
    (255, 255, 255),
    2
)
```

```
cv2.imwrite("opencv_practical.png", img)
```

```
cv2.imshow("Saved Image", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Explain image dimensions using code

Use .shape to get height, width, and channels.

Code

```
img = np.zeros((240, 320, 3), dtype=np.uint8)
```

```
print("Image shape:", img.shape)
```

```
print("Height:", img.shape[0])
```

```
print("Width:", img.shape[1])
```

```
print("Channels:", img.shape[2])
```

Create a random noise image

Random pixel values simulate noise.

Code

```
noise = np.random.randint(0, 256, (300, 300), dtype=np.uint8)
```

```
cv2.imshow("Noise Image", noise)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Class3

Mean vs Median Filtering

Mean Filter

Kernel:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

```
mean_filtered = cv2.blur(noisy, (3,3))
```

- (5,5) → kernel size
- OpenCV performs **convolution**
- It replaces each pixel with the **average of its 5×5 neighborhood**

◆ Mathematical Operation

Mean filter formula:

$$g(x, y) = \frac{1}{25} \sum_{i=-2}^2 \sum_{j=-2}^2 f(x + i, y + j)$$

Kernel:

$$\frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

◆ Step-by-Step Example (Simplified 3×3 Case)

Suppose pixel neighborhood:

$$\begin{bmatrix} 10 & 20 & 30 \\ 20 & 100 & 30 \\ 15 & 25 & 35 \end{bmatrix}$$

Step 1: Add all values

$$10 + 20 + 30 + 20 + 100 + 30 + 15 + 25 + 35 = 285$$

Step 2: Divide by 9

$$285/9 = 31.67$$

New center pixel = 32

- 👉 The large 100 is reduced
- 👉 Image becomes smooth
- 👉 Edges get blurred

➤ Median Filter

What This Line Means

`median_filtered = cv2.medianBlur(gray, 5)`

- 5 → 5×5 window
- It does NOT use convolution
- It sorts the 25 values
- Picks the middle value

◆ Mathematical Operation

$$g(x, y) = \text{median}\{f(x + i, y + j)\}$$

This is a non-linear filter

◆ Step-by-Step Example

Same matrix:

$$\begin{bmatrix} 10 & 20 & 30 \\ 20 & 100 & 30 \\ 15 & 25 & 35 \end{bmatrix}$$

Flatten values:

10, 20, 30, 20, 100, 30, 15, 25, 35

Sort:

10, 15, 20, 20, 25, 30, 30, 35, 100

Median = 25

New center pixel = 25

👉 The 100 (outlier) is removed completely.

Why Median Removes Salt & Pepper Noise Better

Salt noise = 255

Pepper noise = 0

Example:

10, 12, 15, 255, 18

Mean:

$$(10 + 12 + 15 + 255 + 18)/5 = 62$$

Still corrupted.

Median:

Sorted → 10, 12, 15, 18, 255

Median = 15

Perfect restoration.

`median_filtered = cv2.medianBlur(noisy, 3)`

Comparison

Feature	Mean	Median
Removes Gaussian	Yes	Moderate
Removes Salt-Pepper	Poor	Excellent
Edge Preservation	Poor	Better

Sharpening

LAPLACIAN FILTER (Second Derivative)

Mathematical Background

Laplacian = Second order derivative.

Discrete Approximation (3×3 Kernel)

Common kernels:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

OR

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Theory

Second derivative operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Enhancement:

$$g = f - \nabla^2 f$$

- (5,5) → kernel size
- OpenCV performs **convolution**
- It replaces each pixel with the **average of its 5×5 neighborhood**

Sharpening Formula

- $g(x, y) = f(x, y) - \nabla^2 f(x, y)$
- Why subtract?
- Because Laplacian highlights edges (high frequency).
Subtracting enhances sharpness.

◆ Mathematical Operation

Mean filter formula:

$$g(x, y) = \frac{1}{25} \sum_{i=-2}^2 \sum_{j=-2}^2 f(x + i, y + j)$$

Kernel:

$$\frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

◆ Step-by-Step Example (Simplified 3×3 Case)

Suppose pixel neighborhood:

$$\begin{bmatrix} 10 & 20 & 30 \\ 20 & 100 & 30 \\ 15 & 25 & 35 \end{bmatrix}$$

Step 1: Add all values

$$10 + 20 + 30 + 20 + 100 + 30 + 15 + 25 + 35 = 285$$

Step 2: Divide by 9

$$285/9 = 31.67$$

New center pixel = 32

- 👉 The large 100 is reduced
- 👉 Image becomes smooth
- 👉 Edges get blurred

◆ Laplacian Code

```
lap = cv2.Laplacian(img, cv2.CV_64F)
lap = np.uint8(np.absolute(lap))
```

```
sharpened = cv2.subtract(img, lap)
```

```
cv2.imshow("Laplacian", lap)
```

```
cv2.imshow("Sharpened", sharpened)
```

```
cv2.waitKey(0)
```

2 High-Pass Sharpening Kernel# -----

```
kernel = np.array([[ 0, -1, 0], [-1, 5, -1], [ 0, -1, 0]])
```

```
highpass_sharpened = cv2.filter2D(gray, -1, kernel)
```

3 Unsharp Masking#

$g = f + (f - blurred)$

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

```
mask = cv2.subtract(gray, blur)
```

```
unsharp = cv2.add(gray, mask)
```

Complete code

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
img = cv2.imread('Mona_Lisa.jpg')
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# -----
```

```
# 1 Laplacian Sharpening
```

```
#  $g = f - \nabla^2 f$ 
```

```
# -----  
  
laplacian = cv2.Laplacian(gray, cv2.CV_64F)  
laplacian_abs = np.uint8(np.absolute(laplacian))  
lap_sharpened = cv2.subtract(gray, laplacian_abs)
```

```
# -----  
  
# 2 High-Pass Sharpening Kernel
```

```
# -----  
  
kernel = np.array([[ 0, -1, 0],  
                   [-1, 5, -1],  
                   [ 0, -1, 0]])
```

```
highpass_sharpened = cv2.filter2D(gray, -1, kernel)
```

```
# -----  
  
# 3 Unsharp Masking
```

```
#  $g = f + (f - \text{blurred})$ 
```

```
# -----  
  
blur = cv2.GaussianBlur(gray, (5,5), 0)  
mask = cv2.subtract(gray, blur)  
unsharp = cv2.add(gray, mask)
```

```
# -----  
  
# Display using Matplotlib
```

```
# -----  
  
plt.figure(figsize=(12,8))
```

```
plt.subplot(2,2,1)  
plt.imshow(gray, cmap='gray')  
plt.title("Original")  
plt.axis('off')
```

```
plt.subplot(2,2,2)
plt.imshow(lap_sharpened, cmap='gray')
plt.title("Laplacian Sharpened")
plt.axis('off')
```

```
plt.subplot(2,2,3)
plt.imshow(highpass_sharpened, cmap='gray')
plt.title("High-Pass Kernel Sharpened")
plt.axis('off')
```

```
plt.subplot(2,2,4)
plt.imshow(unsharp, cmap='gray')
plt.title("Unsharp Masking")
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```

Edge Detection

Sobel Edge Detection

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

-1	-2	-1
0	0	0
1	2	1

(a)

-1	0	1
-2	0	2
-1	0	1

(b)

-1	0	+1
-2	0	+2
-1	0	+1

(a)

+1	+2	+1
0	0	0
-1	-2	-1

(b)



(c)



(d)

4

✦ First Derivative Operator

$$G = \sqrt{G_x^2 + G_y^2}$$

◆ Code

```
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
```

```
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
```

```
magnitude = cv2.magnitude(sobelx, sobely)
```

```
cv2.imshow("Sobel X", np.uint8(np.absolute(sobelx)))
```

```
cv2.imshow("Sobel Y", np.uint8(np.absolute(sobely)))
```

```
cv2.imshow("Magnitude", np.uint8(magnitude))
```

```
cv2.waitKey(0)
```

Compare: Smoothing vs Sharpening

Aspect	Smoothing	Sharpening
Goal	Remove noise	Enhance edges
Operation	Low-pass filtering	High-pass filtering
Example	Mean, Median	Laplacian
Effect	Blur image	Increase contrast at edges

1 LAPLACIAN EDGE DETECTION

1. Concept

Laplacian is a **second-order derivative operator**.

It detects areas where intensity changes rapidly.

Edges occur at points where:

$$\nabla^2 f \neq 0$$

2. Mathematical Definition

Continuous form:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Discrete approximation (3×3 kernel):

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

OR

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

3. How It Works

For each pixel:

$$g(x, y) = 4f(x, y) - f(x + 1, y) - f(x - 1, y) - f(x, y + 1) - f(x, y - 1)$$

Flat region → output ≈ 0

Edge → large positive or negative value

● 4. Important Property

- No direction information
- Highly sensitive to noise
- Detects zero-crossings

● 5. OpenCV Code

```
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
```

```
laplacian = np.uint8(np.absolute(laplacian))
```

● 6. Advantages

- ✓ Simple
- ✓ Detects thin edges

● 7. Disadvantages

- ✗ Very noise sensitive
- ✗ No edge direction

● 8. Improvement: LoG (Laplacian of Gaussian)

Smooth first:

$$\text{LoG} = \nabla^2(\text{Gaussian} * f)$$

This reduces noise sensitivity.

■ CANNY EDGE DETECTION

● 1. Concept

Canny is a **multi-stage optimal edge detector**.

It produces:

- Thin edges
- Low noise
- Good localization

● 2. Canny Algorithm Steps

◆ Step 1: Gaussian Smoothing

$$f_{smooth} = \text{Gaussian}(f)$$

Removes noise.

◆ Step 2: Gradient Computation

Using Sobel:

$$G_x = \frac{\partial f}{\partial x}$$
$$G_y = \frac{\partial f}{\partial y}$$

Magnitude:

$$G = \sqrt{G_x^2 + G_y^2}$$

Direction:

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

◆ Step 3: Non-Maximum Suppression

Keeps only local maxima along gradient direction.

Produces thin edges.

◆ Step 4: Double Thresholding

Two thresholds:

- High threshold → Strong edges
 - Low threshold → Weak edges
-

◆ Step 5: Edge Tracking (Hysteresis)

Weak edges are kept only if connected to strong edges.

Removes isolated noise edges.

● 3. OpenCV Code

```
canny = cv2.Canny(blur, 100, 200)
```

Where:

- 100 → lower threshold
 - 200 → upper threshold
-

● 4. Why Two Thresholds?

To:

- Reduce false edges
- Preserve connected edges

Typical rule:

$$Low \approx 0.4 \times High$$

● 5. Advantages

- ✓ Thin edges
- ✓ Less noise
- ✓ Good localization
- ✓ Widely used in industry

Class4

Min Max filter

Salt noise → pixel value = 255

Pepper noise → pixel value = 0

◆ Min Filter

$$g(x, y) = \min \{f(x + i, y + j)\}$$

- Removes **salt noise (white dots)**
 - Expands dark regions
-

◆ Max Filter

$$g(x, y) = \max \{f(x + i, y + j)\}$$

- Removes **pepper noise (black dots)**
 - Expands bright regions
-



2 Complete Working Code (With Salt & Pepper Noise)

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Read image

img = cv2.imread('Mona_Lisa.jpg')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


# -----

# Function to add Salt & Pepper Noise

# -----

def add_salt_pepper(image, prob):

    noisy = np.copy(image)

    rows, cols = image.shape
```

```

# Salt noise

num_salt = int(prob * rows * cols / 2)

coords = [np.random.randint(0, i-1, num_salt) for i in image.shape]

noisy[coords[0], coords[1]] = 255


# Pepper noise

num_pepper = int(prob * rows * cols / 2)

coords = [np.random.randint(0, i-1, num_pepper) for i in image.shape]

noisy[coords[0], coords[1]] = 0


return noisy


# Add noise

noisy = add_salt_pepper(gray, 0.05)


# -----

# Min Filter (Erosion)

# -----

kernel = np.ones((3,3), np.uint8)

min_filtered = cv2.erode(noisy, kernel)


# -----

# Max Filter (Dilation)

# -----

max_filtered = cv2.dilate(noisy, kernel)


# -----

# Display

# -----

plt.figure(figsize=(12,8))

```

```
plt.subplot(2,2,1)
```

```
plt.imshow(gray, cmap='gray')
```

```
plt.title("Original")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,2)
```

```
plt.imshow(noisy, cmap='gray')
```

```
plt.title("Salt & Pepper Noise")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,3)
```

```
plt.imshow(min_filtered, cmap='gray')
```

```
plt.title("Min Filter (Removes Salt)")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,4)
```

```
plt.imshow(max_filtered, cmap='gray')
```

```
plt.title("Max Filter (Removes Pepper)")
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

3 Why We Use erode and dilate

In OpenCV:

- `cv2.erode()` → Min filter
- `cv2.dilate()` → Max filter

Because:

Erosion = takes minimum in neighborhood

Dilation = takes maximum in neighborhood

Filter	Effect
--------	--------

Min	Removes white salt noise
-----	--------------------------

Max	Removes black pepper noise
-----	----------------------------

Median	Removes both effectively
--------	--------------------------

Mean → median and median → mean

```
import cv2
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Read image
```

```
img = cv2.imread('Mona_Lisa.jpg')
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# -----
```

```
# Add Salt & Pepper Noise
```

```
# -----
```

```
def add_salt_pepper(image, prob=0.05):
```

```
    noisy = image.copy()
```

```
    rows, cols = image.shape
```

```
    # Salt
```

```
    num_salt = int(prob * rows * cols / 2)
```

```
    coords = [np.random.randint(0, i-1, num_salt) for i in image.shape]
```

```
    noisy[coords[0], coords[1]] = 255
```

```
    # Pepper
```

```

num_pepper = int(prob * rows * cols / 2)

coords = [np.random.randint(0, i-1, num_pepper) for i in image.shape]

noisy[coords[0], coords[1]] = 0


return noisy


noisy = add_salt_pepper(gray, 0.05)


# -----
# 1 Median → Mean
# -----

median_first = cv2.medianBlur(noisy, 5)
median_then_mean = cv2.blur(median_first, (5,5))


# -----
# 2 Mean → Median
# -----

mean_first = cv2.blur(noisy, (5,5))
mean_then_median = cv2.medianBlur(mean_first, 5)


# -----
# Display
# -----

plt.figure(figsize=(12,8))


plt.subplot(2,2,1)
plt.imshow(noisy, cmap='gray')
plt.title("Noisy Image")
plt.axis('off')


plt.subplot(2,2,2)

```

```
plt.imshow(median_then_mean, cmap='gray')  
plt.title("Median → Mean")  
plt.axis('off')
```

```
plt.subplot(2,2,3)  
plt.imshow(mean_then_median, cmap='gray')  
plt.title("Mean → Median")  
plt.axis('off')
```

```
plt.subplot(2,2,4)  
plt.imshow(gray, cmap='gray')  
plt.title("Original")  
plt.axis('off')
```

```
plt.tight_layout()  
plt.show()
```

Median → Mean

- Salt & pepper removed first
- Then smoothing
- Result: Clean + slightly blurred
- Usually better

Mean → Median

- Mean blurs salt & pepper noise
- Median cannot remove blurred noise completely
- Result: Slight residual artifacts

Object scaling logical arithmetic operation

Code: Object Scaling

```
import cv2  
import matplotlib.pyplot as plt
```



```
img = cv2.imread('Mona_Lisa.jpg')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Scaling factors
scale_x = 0.5
scale_y = 0.5

scaled = cv2.resize(img_rgb, None, fx=scale_x, fy=scale_y)

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(scaled)
plt.title("Scaled (0.5x)")
plt.axis('off')

plt.show()
```

● Important Teaching Insight

If scale > 1 → Zoom in

If scale < 1 → Zoom out

Arithmetic Operations

Used for:

- Brightness control
- Image blending
- Masking

◇ 1. Addition (Brightness Increase)

$$g(x, y) = f(x, y) + k$$

```
bright = cv2.add(img, 50)
```

◇ 2. Subtraction (Darkening)

```
dark = cv2.subtract(img, 50)
```

◇ 3. Image Blending

$$g = \alpha f_1 + \beta f_2$$

```
blend = cv2.addWeighted(img, 0.7, scaled, 0.3, 0)
```

Logical Operations (Bitwise)

Used for:

- Object extraction
- Masking
- Background removal

◆ Create Binary Mask

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
_, mask = cv2.threshold(gray, 120, 255, cv2.THRESH_BINARY)
```

◆ 1. AND Operation

$$Result = Image \wedge Mask$$

```
and_img = cv2.bitwise_and(img, img, mask=mask)
```

Used to extract object.

◆ 2. OR Operation

$$Result = A \vee B$$

```
or_img = cv2.bitwise_or(img, img)
```

◆ 3. NOT Operation

$$Result = \neg Image$$

```
not_img = cv2.bitwise_not(img)
```

◆ 4. XOR Operation

$$Result = A \oplus B$$

```
xor_img = cv2.bitwise_xor(img, not_img)
```

● Logical Operation Truth Table (Important for Class)

A B AND OR XOR

0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

● Application Example: Object Extraction

```
object_only = cv2.bitwise_and(img, img, mask=mask)
```

This keeps only foreground object.

Contrast Stretching

1. Concept

Low contrast image:

- Pixel values are concentrated in a small intensity range.
- Image looks dull / faded.

Goal:

Expand intensity range to improve visibility.

2. Mathematical Formula

If:

$$\begin{aligned}r_{min} &= \min (f(x, y)) \\ r_{max} &= \max (f(x, y))\end{aligned}$$

Then contrast stretching:

$$g(x, y) = \frac{f(x, y) - r_{min}}{r_{max} - r_{min}} \times 255$$

This maps:

$$[r_{min}, r_{max}] \rightarrow [0, 255]$$

Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread('Mona_Lisa.jpg', 0)

# Simulate low contrast image
low_contrast = cv2.normalize(img, None, 50, 150, cv2.NORM_MINMAX)
```

```
# Contrast stretching

r_min = np.min(low_contrast)
r_max = np.max(low_contrast)

stretched = (low_contrast - r_min) * (255 / (r_max - r_min))
stretched = np.uint8(stretched)

# Display

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(low_contrast, cmap='gray')
plt.title("Low Contrast")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(stretched, cmap='gray')
plt.title("Contrast Stretched")
plt.axis('off')

plt.subplot(1,3,3)
plt.hist(low_contrast.ravel(), 256, [0,256])
plt.title("Histogram (Low Contrast)")

plt.tight_layout()
plt.show()
```

Histogram

● 1. Definition

Histogram shows:

Frequency distribution of gray levels.

$$p(r_k) = \frac{n_k}{MN}$$

Where:

- n_k = number of pixels with intensity r_k
 - $M \times N$ = total pixels
-

● 2. Why Important?

Histogram tells:

- Bright image → right side
 - Dark image → left side
 - Low contrast → narrow distribution
-

Code: Histogram Plot

```
plt.figure(figsize=(6,4))  
plt.hist(img.ravel(), 256, [0,256])  
plt.title("Histogram")  
plt.xlabel("Intensity")  
plt.ylabel("Frequency")  
plt.show()
```

Histogram Equalization

● 1. Concept

Improves contrast by redistributing intensities.

Uses Cumulative Distribution Function (CDF).

● 2. Mathematical Formula

Step 1: Compute probability

$$p(r_k) = \frac{n_k}{MN}$$

Step 2: Cumulative sum

$$CDF(r_k) = \sum_{j=0}^k p(r_j)$$

Step 3: Transform

$$s_k = (L - 1) \times CDF(r_k)$$

Where:

- $L = 256$ for 8-bit image
-

● 3. Effect

- Makes histogram more uniform
 - Enhances global contrast
 - Non-linear transformation
-

Code: Histogram Equalization

```
equalized = cv2.equalizeHist(low_contrast)
```

```
plt.figure(figsize=(12,4))
```

```
plt.subplot(1,3,1)
```

```
plt.imshow(low_contrast, cmap='gray')  
plt.title("Low Contrast")  
plt.axis('off')
```

```
plt.subplot(1,3,2)  
plt.imshow(equalized, cmap='gray')  
plt.title("Histogram Equalized")  
plt.axis('off')
```

```
plt.subplot(1,3,3)  
plt.hist(equalized.ravel(), 256, [0,256])  
plt.title("Equalized Histogram")
```

```
plt.tight_layout()  
plt.show()
```


What is Contrast Enhancement?

Contrast enhancement improves visibility by modifying pixel intensities.

It is done using:

$$g(x, y) = T(f(x, y))$$

Where:

- $f(x, y)$ → original image
- $g(x, y)$ → enhanced image
- T → transformation function

These are **point processing operations** (operate pixel-by-pixel).

Linear Transformation (Contrast Stretching)

Mathematical Formula

$$g(x, y) = \frac{f(x, y) - r_{min}}{r_{max} - r_{min}} \times 255$$

Maps intensity range to full dynamic range.

Code

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

img = cv2.imread('Mona_Lisa.jpg', 0)

# Create low contrast image

low = cv2.normalize(img, None, 80, 150, cv2.NORM_MINMAX)

r_min = np.min(low)

r_max = np.max(low)
```

```
stretched = (low - r_min) * (255 / (r_max - r_min))
```

```
stretched = np.uint8(stretched)
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
```

```
plt.imshow(low, cmap='gray')
```

```
plt.title("Low Contrast")
```

```
plt.axis('off')
```

```
plt.subplot(1,2,2)
```

```
plt.imshow(stretched, cmap='gray')
```

```
plt.title("Contrast Stretched")
```

```
plt.axis('off')
```

```
plt.show()
```

◆ 2 Log Transformation

Used to enhance dark regions.

Formula:

$$g = c \log(1 + f)$$

Enhances low-intensity pixels.

Code

```
c = 255 / np.log(1 + np.max(img))
```

```
log_transform = c * np.log(1 + img)
```

```
log_transform = np.uint8(log_transform)
```

◆ 3 Power-Law (Gamma) Transformation

$$g = cf^\gamma$$

Where:

- $\gamma < 1 \rightarrow$ brighten image
- $\gamma > 1 \rightarrow$ darken image

Code

```
gamma = 0.5 # Try 0.5 and 2.0
```

```
gamma_corrected = np.array(255*(img/255)**gamma, dtype='uint8')
```

Histogram Equalization

Non-linear transformation using CDF.

$$s_k = (L - 1) \times CDF(r_k)$$

Code

```
equalized = cv2.equalizeHist(img)
```

Comparison Table

Method	Type	Effect
Contrast Stretching	Linear	Expands range
Log	Non-linear	Enhances dark areas
Gamma	Non-linear	Brightness control
Histogram Equalization	Non-linear	Uniform histogram

Graphical Interpretation

Transformation function $T(r)$:

- Linear $\rightarrow$ straight line
- Log $\rightarrow$ compresses high intensities
- Gamma $\rightarrow$ curved function

Gray Level Transformation (GLT)

Types of Gray Level Transformations

◆ 1 Image Negative

Used for enhancing white details in dark regions.

Formula:

$$g(x, y) = L - 1 - f(x, y)$$

For 8-bit image:

$$g = 255 - f$$

📄 Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread('Mona_Lisa.jpg', 0)

negative = 255 - img

plt.imshow(negative, cmap='gray')
plt.title("Image Negative")
plt.axis('off')
plt.show()
```

◆ 2 Log Transformation

Enhances dark pixels.

$$g = c \log(1 + f)$$

Where:

$$c = \frac{255}{\log(1 + \max(f))}$$

Code

```
c = 255 / np.log(1 + np.max(img))  
log_img = c * np.log(1 + img)  
log_img = np.uint8(log_img)
```

3 Power-Law (Gamma) Transformation

Controls brightness.

$$g = cf^\gamma$$

- $\gamma < 1 \rightarrow$ brighten
 - $\gamma > 1 \rightarrow$ darken
-

Code

```
gamma = 0.5  
gamma_img = np.array(255*(img/255)**gamma, dtype='uint8')
```

4 Contrast Stretching (Linear Transformation)

$$g = \frac{f - r_{\min}}{r_{\max} - r_{\min}} \times 255$$

Code

```
r_min = np.min(img)  
r_max = np.max(img)  
  
stretch = (img - r_min) * (255 / (r_max - r_min))  
stretch = np.uint8(stretch)
```

◆ 5 Piecewise Linear Transformation

Used to emphasize certain intensity ranges.

Example:

Contrast stretching with thresholds:

$$g = \begin{cases} 0 & f < r_1 \\ \text{linear scaling} & r_1 \leq f \leq r_2 \\ 255 & f > r_2 \end{cases}$$

● Graphical Representation

Transformation function $T(r)$:

- Straight line → linear
- Log curve → dark enhancement
- Power curve → gamma correction
- Inverted line → negative

● Important Concepts (For M.Tech)

1 Point Processing

Each output pixel depends only on its corresponding input pixel.

2 Dynamic Range

Range between minimum and maximum intensity.

3 Brightness vs Contrast

Brightness → shift curve

Contrast → stretch curve

● Combined Demonstration Code

```
plt.figure(figsize=(12,8))
```

```
plt.subplot(2,2,1)
```

```
plt.imshow(img, cmap='gray')
```

```
plt.title("Original")
```

```
plt.axis('off')
```

```
plt.subplot(2,2,2)
plt.imshow(negative, cmap='gray')
plt.title("Negative")
plt.axis('off')
```

```
plt.subplot(2,2,3)
plt.imshow(log_img, cmap='gray')
plt.title("Log Transform")
plt.axis('off')
```

```
plt.subplot(2,2,4)
plt.imshow(gamma_img, cmap='gray')
plt.title("Gamma Transform")
plt.axis('off')
```

```
plt.tight_layout()
plt.show()
```

● Comparison Table

Transformation Type		Purpose
Negative	Linear	Highlight white details
Log	Non-linear	Enhance dark regions
Gamma	Non-linear	Brightness control
Stretching	Linear	Expand contrast